Interfacet java.lang.Comparable
Canonical class in Java

Interface?

- Interface = gränssnitt
- I Java syftar nyckelordet ”interface” på en deklaration av en uppsättning publika och *abstrakta* metoder
- Inga datamedlemmar och ingen implementation i ett Java-interface
  - förutom klasskonstanter, static final
  - numera tillåtet med ”default-implementation” av metoder
- ```
  public interface Printable {

        public abstract void print(Printer printer);

      // possibly more abstract methods...

  }
  ```
- En klass kan implementera ett eller flera interface (men endast ärva från en superklass)
  ```

Interfacet Comparable - för rangordning

- Om vi vill kunna sortera objekt måste vi definiera en rangordning för objekten
  - I språket C kan man använda funktionen strcmp(str1, str2) för att rangordna textsträngar
- I Javas API definieras rangordning mellan två objekt via interfacet java.lang.Comparable, med metoden compareTo
  - Implementera interfacet i klasser där det är meningsfullt att rangordna objekt, t.ex. för sortering
- Interfacet är generiskt, dvs har en typparameter:
  - ```
    public interface Comparable<T> {
        public abstract int compareTo(T other);
    }
    ```
    ```

Interfacet Comparable - för rangordning

- Exempel på anrop av metoden:

```
if(a.compareTo(b) > 0) { // a is ”greater” than b
```



- Kontrakt för metoden compareTo (jämför med strcmp i C):

Returnera	
< 0 om	*this* anses < *other*
0 om	*this* anses lika med *other*
> 0 om	*this* anses > *other*

Interfacet Comparable

- Ett exempel på implementation av interfacet Comparable<T>
- Klassen Integer är en s.k. wrapper class, med en int som data, som tillåter oss att hantera heltal som objekt
- Rangordning: storleken på value
  Låt compareTo returnera skillnaden mellan värdena
- Vi kan nu jämföra objekt med t.ex.
  Integer a = new Integer(42);
  Integer b = new Integer(47);
  if(a.compareTo(b) > 0) {
    // a is greater

```
public class Integer implements Comparable<Integer> {

    private int value;

    public Integer(int value) {
        this.value = value;
    }

    public int getValue() {
        return value;
    }

    @Override
    public int compareTo(Integer other) {
        return this.value - other.value;
    }

    @Override
    public String toString() {
        return "value=" + value;
    }
    …
}
```

Interfacet java.lang.Comparable

- Vi kan ha flera ”nycklar” när vi rangordnar i compareTo
- Här: Jämför först titeln, om titlarna är lika jämför istället datumet
- Många klasser i Javas API implementerar Comparable, t.ex. String och LocalDate
- Vi kan utnyttja detta i vår implementation av compareTo

```
public class TextDocument
        implements Printable, Comparable<TextDocument> {
    private String title;
    private LocalDate lastUpdated;

    public TextDocument(String title, LocalDate lastUpdated) {
        this.title = title;
        this.lastUpdated = lastUpdated;
    }

    @Override
    public int compareTo(TextDocument other) {
       // first key, title, use compareTo in class String
       int result = this.title.compareTo(other.title);
       if (result == 0) {
            // second  key, lastUpdated
            result =
                this.lastUpdated.compareTo(other.lastUpdated); }
        return result;
    }

    …
}
```

Polymorf sortering via interfacet Comparable

- *Argumenttypen är Comparable[]…*
- *… vilket innebär vi kan anropa med en array av vilken typ som helst som implementerar interfacet Comparable*
- I anropet arr[j].compareTo(arr[…]) bestämmer typen på det objekt arr[j] refererar vilken version av compareTo som exekveras …
- … polymorf kod, dynamisk bindning vid anropet av metoden

```
public static void sortObjects(Comparable[] arr) {
    int minindex;
    Comparable temp;

    for (int i = 0; i < arr.length - 1; i++) {
        minindex = i;
        for (int j = i + 1; j < arr.length; j++) {

            if (arr[j].compareTo(arr[minindex]) < 0) {
                minindex = j;
            }
        }
        // Swap
        temp = arr[i];
        arr[i] = arr[minindex];
        arr[minindex] = temp;
    }
}
```

Polymorf sortering via interfacet Comparable

- Vi behöver alltså bara skriva en enda sorteringsmetod …
- … som kan sortera objekt av alla typer (klasser) som implementerar interfacet Comparable
  - Egna klasser som Integer, TextDocument, …
  - API-klasser som String, LocalDate, … - läs dokumentationen för respektive klass

- ```
  Integer[] ints = new Integer[3];
  ints[0] = new Integer(42);
  ints[1] = new Integer(13);
  ints[2] = new Integer(-1);

  Sort.sortObjects(ints);
  ```

- ```
  TextDocument[] docs = new TextDocument[3];
  docs[0] = new TextDocument("OOP basics", LocalDate.of(2025, 1, 17));
  docs[1] = new TextDocument("Inheritance", LocalDate.of(2023, 1, 24));
  docs[2] = new TextDocument("Inheritance", LocalDate.of(2023, 1, 27));

  Sort.sortObjects(docs);
  ```

Polymorf sortering av Comparables

- Motsvarande metod finns (naturligtvis) redan i Javas API…
- För arrayer
  - `java.util.Arrays.sort(arrayOfComparables)`
  - klassmetod, static
- Listor som ArrayList m.fl.
  - `java.util.Collections.sort(listOfComparables)`
  - klassmetod, static

Canonical form of a Java class

A canonical form (following well-established rules) of a Java class should provide the following

- methods for testing for object equality: equals and hashCode
- a method, toString, for String representation
- if meaningful for this type: an implementation of compareTo from interface Comparable defining the natural ordering for objects of this type
  - equals should return true if, and only if, compareTo returns 0
- If needed, a method for cloning objects
  - if necessary, making a deep copy
  - Personally, I do not recommend overriding Object.clone

Canonical form of a Java class

- equals should compare *objects* for equality
  - a == b only returns true if a and b references the *same* object
  - equals should return true if, and only if, compareTo returns 0

```
public class TextDocument implements Printable, Comparable<TextDocument> {
    …
    @Override
    public int compareTo(TextDocument other) {
        int result = this.title.compareTo(other.title);
        if (result == 0) {
            result =
                this.lastUpdated.compareTo(other.lastUpdated);
        }
        return result;
    }

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (other instanceof TextDocument) {
            TextDocument otherDoc = (TextDocument) other;
            return title.equals(otherDoc.title) &&
                   lastUpdated.equals(otherDoc.lastUpdated);
            // alternative: return this.compareT(otherDoc) == 0;
        }
        return false;
    }

    @Override
    public int hashCode() {
          return Objects.hash(title, lastUpdated);
    }
```